

Slot storage

Each group below, plus OZ Initializable and ReentrancyGuard, has its own ERC-7201 location. Blue = proposed rename, not in code.

LOCATION · STRUCT	FIELD	TYPE	HOLDS	WRITTEN BY	PROPOSED
slots.settings Settings → Governance How the slot is governed. Fixed at creation except manager. Renamed so “settings” means only the module’s config.	currency	IERC20	Payment token. Zero address = native ETH.	initialize	
	manager	address	Proposes terms, accepts module updates, hands over. Set exactly when something is mutable.	initialize · setManager	
	mutableTax · mutableRecipient · mutableModule	bool ×3	Which terms may ever change: tax rate and runway · recipient (and raising a module fee) · module and its scopes.	initialize	
slots.terms.tax TaxTerms · live	recipient	address	Receives collected tax, minus the module fee.	initialize · terms land	
	rateBps	uint16	Tax per 30 days, in basis points of the price. 1–10,000.	initialize · terms land	
	minRunwaySeconds	uint32	A buyer’s deposit must cover this long at their price. ≤ 365 days.	initialize · terms land	
slots.module InstalledModule One record. Installing copies one in; removing deletes it. Same shape in Pending.module.	target	address	The installed module. Zero = none.	initialize · module lands	module
	scopes	uint16	Copied from the module. beforeBuy 1 · beforeSelfAssess 2 · afterBuy 4 · afterRelease 8 · afterLiquidate 16 · afterSettle 32 · strict 64 · onInstall 128 · onUninstall 256.	initialize · module or scopes land	strict → afterCallbacks-MustSucceed
	fee	ModuleFee	Copied from the module: bps (0–10,000 of collected tax) and recipient. Payouts use it.	initialize · acceptFee · module lands	
	settings	bytes	abi.encode of the fields the module’s definition lists. Handed to every callback.	initialize · module lands	
slots.pending Pending Stored exactly as pending() returns it. Lands at the next buy or applyTerms, 1 hour after proposal.	taxTerms	TaxTerms	Proposed tax values. Only the fields named in mask count.	proposeTerms · cancel · land	
	module	InstalledModule	A proposed module with the scopes and fee reviewed for it, or only newly accepted scopes. Checked again when it lands.	proposeTerms · acceptScopes · cancel · land	
	mask	uint16	TAX_RATE 1 · RECIPIENT 2 · MIN_RUNWAY 4 · MODULE 8 · SCOPES 16 (never with MODULE).	propose · acceptScopes · cancel · land	
	proposedAt	uint64	Start of the delay. Any new proposal restarts it.	proposeTerms · acceptScopes	
slots.occupancy Occupancy Who holds the slot, at what price, funded by how much.	occupant	address	Current holder. Zero = vacant.	buy · release · liquidate	
	since	uint64	When the current tenure began.	buy	occupiedSince matches the view
	tenureId	uint64	Counts tenures. Scopes operator approvals to one tenure.	buy	
	lastSettled	uint64	Tax is accounted up to this timestamp.	every settle	
	taxCarry	uint64	Remainder below one unit of currency, carried to the next settle.	every settle · buy · vacate	
	price	uint256	Self-assessed price. ≤ 2 ¹²⁸ –1.	buy · selfAssess	
	deposit	uint256	Escrow that tax is drawn from.	buy · topUp · withdraw · settle	
slots.ledger Ledger Tax taken, and what could not be paid either way.	collectedTax	uint256	Tax taken from deposits, not yet paid out.	settle · collect · any payout	
	withdrawableOf	address → uint256	Payouts that could not be pushed. Taken with claim.	failed payout · claim	claimableOf withdraw is the deposit
	debtOf	address → uint256	Tax owed beyond an emptied deposit. Repaid first by topUp or buy.	settle · topUp · buy	
	operatorOf	tenure → addr → bool	Who may reprice for the occupant, for one tenure only.	setOperator	

Terms land at the next buy or applyTerms, 1 hour after proposal. Settle runs first in almost every call: tax moves from deposit to collectedTax; a shortfall becomes debt. Upgrades append fields only.

Slot methods · writes

Grouped by who may call, then what the slot calls on its module.
Blue = proposed rename, not in code.

CALLER	METHOD	DOES	PROPOSED
Occupant The current holder. selfAssess also takes an operator.	selfAssess (newPrice)	Restate the price; tax follows from now on. The module may veto.	
	withdraw (amount)	Take deposit back, keeping what the current terms require.	
	release ()	Leave. The rest of the deposit is refunded.	
	setOperator (operator, allowed)	Let someone reprice on your behalf, for this tenure only.	
	applyTerms ()	Land ripe terms without waiting for a buy. Anyone’s call once the slot is vacant.	
Anyone	buy (account, selfAssessedPrice, depositAmount, maxPayment) PAYABLE	Pay the sitting price (nothing if vacant) plus a deposit, and seat account. Lands ripe terms; the module may veto.	
	topUp (amount) PAYABLE	Add to the occupant’s deposit. Repays their debt first.	
	liquidate ()	Evict an occupant whose deposit is empty. No bounty; the reward is a vacant slot.	
	collect ()	Pay out collected tax: the module fee, then the recipient.	
	claim (account)	Send account a payout that could not be pushed to them.	
	multicall (bytes[] data)	Several calls in one transaction, e.g. liquidate then buy.	
Manager Only on a slot created with something mutable.	proposeTerms (taxTerms, moduleTerms, mask)	Queue changes; they wait 1 hour. Checks the module’s settings and keeps the scopes and fee it reviewed.	
	cancelTerms (mask)	Drop the named queued changes, leaving the rest.	
	acceptFee (ModuleFee expected)	Take the fee the module declares now. Applies at once; a rise needs mutableRecipient.	
	acceptScopes (uint16 expected)	Take the scopes the module declares now. Queued for the next buy; needs mutableModule.	
	setManager (next)	Hand the slot to another manager, immediately.	
Factory Once.	initialize (SlotInit)	Set governance and terms, check the module’s settings, copy its scopes and fee, call onInstall.	
Slot → module ISlotModule. Only callbacks named in the slot’s scopes are called.	checkSettings (bytes settings) VIEW	Revert if the settings are invalid. At proposal and install.	<a>validateSettings
	scopes (bytes settings) VIEW	The callbacks it asks for. May depend on the settings. At proposal, install and acceptScopes.	
	fee (bytes settings) VIEW	Its share of the tax and who receives it. At proposal, install and acceptFee.	
	beforeBuy · beforeSelfAssess (ctx) VIEW	Veto by reverting. Uncapped gas.	
	afterBuy · afterRelease · afterLiquidate · afterSettle (ctx)	Told after the fact. 500k gas; a revert is ignored unless strict.	<a>ctx.owed/paid → taxOwed/taxPaid
	onInstall (ctx)	Attached. A revert is ignored unless strict.	
	onUninstall (ctx)	Being replaced or removed. Always capped, a revert always ignored.	
	definition () OPTIONAL	JSON for UIs (IDescribedModule). The slot never calls it.	<a>uiMetadata IModuleMetadata

Slot methods · reads

All view. Grouped by what they read.
Blue = proposed rename, not in code.

READS	METHOD	RETURNS	DOES	PROPOSED
Terms	<code>terms()</code>	Terms	Tax terms and module terms in force, together.	
	<code>taxTerms()</code>	TaxTerms	Recipient, rate and runway in force.	
	<code>moduleTerms()</code>	ModuleTerms	Installed module and its settings.	
	<code>recipient()</code>	address	Who receives collected tax, minus the module fee.	
	<code>taxRateBps()</code>	uint256	Tax per 30 days, in basis points of the price.	
	<code>minRunwaySeconds()</code>	uint256	How long a buyer’s deposit must last at their price.	
Module	<code>module()</code>	address	Installed module. Zero = none.	
	<code>scopes()</code>	Scopes	The slot’s copy of its scopes, unpacked to booleans.	
	<code>fee()</code>	ModuleFee	The slot’s copy of its fee: bps and recipient.	
Queue	<code>pending()</code>	Pending	Everything queued for the next buy, as stored.	
	<code>hasRipeTerms()</code>	bool	Whether the queue lands at the next buy.	
Governance	<code>currency()</code>	IERC20	Payment token. Zero = native ETH.	
	<code>manager()</code>	address	Who proposes terms and accepts module updates.	
	<code>mutableTax()</code>	bool	Tax rate and runway may change.	
	<code>mutableRecipient()</code>	bool	Recipient may change; a module fee may rise.	
	<code>mutableModule()</code>	bool	Module may be replaced; new scopes may land.	
Occupancy	<code>occupant()</code>	address	Current holder. Zero = vacant.	
	<code>price()</code>	uint256	Self-assessed price.	
	<code>deposit()</code>	uint256	Escrow that tax is drawn from.	
	<code>occupiedSince()</code>	uint64	When the current tenure began.	
	<code>tenureId()</code>	uint64	Counts tenures; operator approvals are scoped to one.	
	<code>lastSettled()</code>	uint64	Tax is accounted up to this timestamp.	
Health	<code>isVacant()</code>	bool	Nobody is seated.	
	<code>isInsolvent()</code>	bool	The deposit can no longer cover what is owed.	
	<code>taxOwed()</code>	uint256	Tax accrued since the last settle.	
	<code>secondsUntillLiquidation()</code>	uint256	Until the deposit runs out. Max if never.	
	<code>isOperator(operator)</code>	bool	May act for the current occupant.	
Ledger	<code>collectedTax()</code>	uint256	Tax taken, not yet paid out.	
	<code>withdrawableOf(account)</code>	uint256	Payouts waiting for claim.	<code>claimableOf</code>
	<code>debtOf(account)</code>	uint256	Tax owed beyond an emptied deposit.	
Quotes	<code>quoteBuy(account, depositAmount)</code>	uint256	Exactly what buy charges: price + deposit + debt.	
	<code>minDepositForBuy(price)</code>	uint256	Smallest deposit buy accepts, under ripe queued terms.	
	<code>minDepositToHold(price)</code>	uint256	Deposit floor under the terms in force.	
Bundles	<code>getSlotInfo()</code>	SlotInfo	Everything above, in one call.	
	<code>getSlotConstants()</code>	SlotConstantsInfo	The constants below, in one call. The only way to read them.	
	<code>version()</code>	uint64	Implementation version.	
SlotLens	<code>moduleUpdate(slot)</code>	ModuleUpdate	On the lens: what the module declares beside the slot’s copy, and whether acceptFee / acceptScopes would change it.	

CONSTANTS · INTERNAL, READ THROUGH GETSLOTCONSTANTS()

MAX_PRICE	2 ¹²⁸ −1	MAX_TAX_BPS	10,000	MAX_MIN_RUNWAY	365 days	TERMS_DELAY	1 hour	BASIS_POINTS	10,000	MONTH	30 days
MODULE_GAS	500,000	PAYOUT_GAS	30,000	TERM_* bits				TAX_RATE 1 · RECIPIENT 2 · MIN_RUNWAY 4 · MODULE 8 · SCOPES 16			
	MODULE_CALLBACK_GAS_LIMIT		NATIVE_PAYOUT_GAS_LIMIT								